

VendorPulse — Why Each Component Is Required

A justification reference for handling client change requests

*Purpose: when the client proposes removing or swapping a component, this gives you the rationale to push back with confidence — or to agree where it genuinely is flexible. For each component: what it does, why it's required, what breaks if it's removed, and the response to give. The two items flagged for deeper discussion — the **relational database** and the **AI Service (LLM integration)** — have their own sections.*

How to use it: lead with the one-line justification, then escalate to "what breaks" only if pushed. Where something is genuinely negotiable, say so — it buys credibility for the parts that aren't.

Quick reference

Component	Required?	One-line justification	If the client wants to cut/change it
Relational DB (PostgreSQL)	Required	Transactional state, integrity, queryable audit — storage can't do these	See §1. Azure SQL is an acceptable swap; Blob/flat files are not
AI Service (LLM integration)	Required	Thin internal service calling Foundry GPT-4o directly; draft text only, behind the approval gate	See §2. MAF SDK was evaluated and removed — fewer dependencies, simpler review
Application Gateway + WAF	Required (WAF non-negotiable)	One hardened, inspected way in; no public backend	Front Door not used — access is internal-only; see §3
Two-VM split (App / Backend)	Recommended	Separates the user-facing tier from data/integration	Can collapse to one VM to save cost, with reduced isolation
Entra OIDC + RBAC	Required	Shell identity, server-side authorization	Not negotiable — it's Shell's own standard
Approval gate (HITL)	Required	The core governance control; AI never acts alone	Not negotiable — it's the product's safety model
WorkflowEngine (12-state)	Required	Deterministic, auditable process; no skipped steps	Not negotiable — it's the deterministic core
Key Vault + Managed Identity	Required	No secrets in code; Shell compliance	Not negotiable
Blob Storage	Required (for files)	Transcripts/minutes; immutable records	Keep — but it is not a database substitute
App Insights + OTel	Required	Observability + immutable audit trail	Not negotiable for a governed AI system
Egress proxy	Required	Shell's controlled, logged outbound	Shell network policy — not ours to remove
Azure AI Foundry in-tenant	Required	Data residency — AI runs inside Shell	Not negotiable for the data-residency story

1. The relational database (Azure PostgreSQL) — required

Short answer to give: "Yes, an RDBMS is required. I'll walk through the rationale, but in one line: VendorPulse's data is transactional, related, queried, and audited — and object storage or flat files can't do any of those safely."

What it does: holds workflow state, user data, approval records, and audit logs — the system's source of truth.

Why it's required — four things only a database gives you:

Transactions and concurrency. The 12-state workflow and the approval gate must update state atomically, and multiple coordinators will act at once. A database gives row-level locking and ACID transactions so two simultaneous actions can't corrupt or lose state. Files/Blob are last-write-wins — one update silently overwrites another.

Integrity and relationships. Requests, approvals, attendees, and audit records are related. The database enforces those relationships (foreign keys, constraints) so the data can't drift into an invalid shape. Storage has no concept of this.

Querying and reporting. Dashboards and status views ("everything pending approval", "this sponsor's history") are single queries against indexed columns. Without a database you'd load and scan every record in application code — slow and unscalable.

A queryable, tamper-evident audit trail. Approval and audit records must be searchable and provable for IRM. A database makes the audit indexed and queryable; combined with the immutable Log Analytics mirror, it satisfies the compliance ask.

What breaks if it's removed: lost or corrupted state under concurrent use; no referential integrity; no reporting without scanning everything; a weak, hard-to-search audit trail. For a governance product these are not acceptable.

The tables this creates (the relational schema): the data normalises into ~16 related tables, each tied back to a cycle by foreign key — which is precisely why a relational store is required:

Table	Purpose	Key relationships
vendor	Vendor master data	—
cycle	One governance cycle; holds the 12-state value	FK → vendor
user	Internal Shell users and roles	—
attendee	Per-cycle attendees (internal + external)	FK → cycle, optional FK → user
slot_proposal	Ranked candidate meeting slots	FK → cycle
meeting	The scheduled meeting / Teams event	FK → cycle, slot_proposal
scorecard_template	Versioned KPI template per cycle	FK → cycle
kpi	KPIs/parameters under a template (4 categories)	FK → scorecard_template
scorecard	The compiled, locked cycle scorecard	FK → cycle
score_submission	Individual KPI scores (vendor + internal)	FK → scorecard, kpi
alignment	Internal alignment record	FK → cycle
vendor_prep	Vendor-prep record	FK → cycle
action_item	Consolidated action register	FK → cycle, owner (user)
meeting_note	Captured notes by type (5 types)	FK → meeting
minutes	Approved meeting minutes	FK → meeting
agent_run	Audit log of every AI Service run / outbound action, with correlation IDs	FK → cycle

(Full relationships are in the ER diagram — see the Client Walkthrough appendix and VendorPulse_Database_ERD.pdf.)

Where we can flex (say this to show reasonableness):

- Azure SQL Database is an acceptable alternative** if that's Shell's data-platform standard — same guarantees, different engine. The application accesses storage through a single repository layer, so switching engines is a contained change, not a rewrite.
- Cost is addressable without dropping the DB:** a serverless (Azure SQL) or Burstable (PostgreSQL) tier scales down between cycles.

Where we cannot flex: replacing the database with **Blob Storage, flat files, or a document/NoSQL store** for the core state. Blob is the right home for *files* (transcripts, minutes) and we already use it there — but it is not a transactional system of record.

One-liner: "Blob is for files; the workflow state, approvals and audit need a database. We can align on Postgres vs Azure SQL to your standard, and tune the tier for cost — but dropping the relational store would reintroduce concurrency, integrity and reporting problems we deliberately engineered out."

2. The AI Service (LLM integration) — required; MAF SDK removed

Short answer to give: "AI is handled by a small in-house AI Service that calls Azure AI Foundry (GPT-4o) directly. It only drafts text, in-tenant, behind the human-approval gate. We evaluated the Microsoft Agent Framework (MAF) and removed it — for our needs it added a fast-moving dependency without enough benefit."

What it does: the AI Service is the single internal component that talks to the model. It wraps Azure AI Foundry (GPT-4o) via the Azure OpenAI SDK, builds grounded prompts from deterministic data, and returns **drafted text** (summaries, briefs, minutes, suggestions). It never computes figures and never sends anything — every output is a draft for the approval gate.

Why a thin AI Service (and why MAF was removed):

- Fewer dependencies, simpler security review.** A small, readable service making a direct, well-understood API call is easier for Shell to review than a fast-churning agent framework (MAF's API changed repeatedly across releases).
- We don't need an agent loop.** Our AI features are single structured calls — "given this context, draft this text" — not multi-step autonomous reasoning. A tool-calling agent framework is more machinery than the workload needs.
- No governance control is lost.** The approval gate, the deterministic core, content-safety (Foundry-level filters) and audit logging live in our own code / the platform regardless of MAF — so removing MAF removes a dependency, not a control.
- Provider stays abstracted.** The model sits behind an LLMPProvider abstraction (config-driven), so GPT-4o can be upgraded or swapped without touching application code.

What breaks if it's removed entirely: no AI drafting — the workflow still runs deterministically (the AI is optional by design), but coordinators lose the drafted briefs, minutes and summaries.

Where we can flex:

- Exact model** (GPT-4o / GPT-4.1) is a config choice — pin a GA model.
- MAF can be reintroduced later** if we ever need true multi-step agents or Foundry Hosted Agents — the AI Service boundary makes that a contained change. Not needed for the MVP.

One-liner: "AI is a thin in-house service calling Foundry directly — draft text only, human-approved, in-tenant. We dropped the agent framework because our calls are single-shot, not agentic; it was a chummy dependency that added no control we don't already own."

3. The rest of the stack — why each is required

Edge: Application Gateway + WAF. The single hardened, inspected way in — TLS termination, OWASP rules, origin-lock so no VM is reachable directly, and load balancing. Because all users are internal, we use **Application Gateway + WAF alone — Azure Front Door is not used** (no public-internet CDN needed). *If removed:* the backend is exposed and unprotected against common web attacks. The **WAF is non-negotiable**.

Two-VM split (App Server / Backend Services). Separates the user-facing tier (auth, workflow, UI) from the data and integration tier (database, Graph, AI). *If removed:* a single VM works but couples the tiers and reduces isolation. *Where we can flex:* the two VMs can be collapsed into one to save cost in lower environments, accepting reduced separation — a reasonable trade to discuss.

Entra OIDC + RBAC. Users sign in with Shell identity; their role is carried in the token and enforced server-side. *If removed:* no governed access control. *Not negotiable* — it's Shell's own identity standard.

Approval gate (HITL). Every AI output is a draft held until a human approves; the workflow blocks until then. *If removed:* the AI could drive actions unreviewed — the one thing the product exists to prevent. *Not negotiable*.

WorkflowEngine (12-state). The deterministic, forward-only state machine that drives every request. *If removed:* the process could be skipped or reordered. *Not negotiable* — it is the deterministic core.

Key Vault + Managed Identity. All secrets (LLM key, Graph cert, JWT keys) fetched at runtime via a password-less Azure identity. *If removed:* secrets end up in code/config — a direct compliance failure. *Not negotiable*.

Blob Storage. Stores transcripts and minutes, immutable once written. *If removed:* nowhere to keep large file artifacts. *Keep* — but note it is for files, not the database.

App Insights + Log Analytics (OpenTelemetry). Observability and the immutable audit trail of every action. *If removed:* no operational visibility and no provable audit. *Not negotiable* for a governed AI system.

Egress proxy. The single, governed, logged outbound path for Graph and Foundry. *If removed:* uncontrolled outbound traffic. *Not ours to remove* — it's Shell's network policy.

Azure AI Foundry in-tenant (GPT-4o). The model runs inside Shell's own tenant, so data sent to it never leaves the boundary. *If removed / replaced with a public API:* breaks the data-residency story. *Not negotiable* for that reason.

What we're genuinely flexible on (lead with this to build trust)

- bullet **Postgres vs Azure SQL** — align to Shell's data-platform standard.
- bullet **Database tier** — serverless/burstable to control cost between cycles.
- bullet **Edge** — Application Gateway + WAF (Front Door not used, as access is internal-only; the WAF stays either way).
- bullet **One VM vs two** — collapse the tiers in non-prod to save cost.
- bullet **Exact GPT model** — config-driven; we standardise on a GA model and can re-evaluate.

What we will not compromise on (and why)

- bullet **A relational database** for state and audit (integrity, concurrency, reporting).
- bullet **The human-approval gate** and the **deterministic WorkflowEngine** (the governance core).
- bullet **Key Vault + Managed Identity, Entra OIDC + RBAC, the WAF, OTel audit, and in-tenant Foundry** (Shell's own compliance and data-residency requirements).

Closing line: "We've separated what's genuinely a preference — the database engine, the tier, how many VMs — from what protects you: the approval gate, the deterministic core, identity, secrets, audit, and keeping the AI inside your tenant. We'll happily flex on the first list; the second list is what makes this pass your own IRM controls."